TABLE OF CONTENTS

ABSTRACT **3**

CHAPTER 1: INTRODUCTION **4**

1.1 INTRODUCTION 5

1.2 BACKGROUND 5

1.3 PROBLEM STATEMENT 6

1.4 PROJECT SCOPE AND RELEVANCE 6

1.5 OBJECTIVES OF THE PROPOSED SYSTEM 7

1.6 EXISTING SYSTEM 7

1.7 PROPOSED SYSTEM 7

1.8 ADVANTAGES 7

CHAPTER 2: SYSTEM ANALYSIS **9**

2.1 INTRODUCTION 10

2.2 EXISTING SYSTEM 10

2.2.1 LIMITATIONS OF EXISTING SYSTEM 10

2.3 PROPOSED SYSTEM 10

2.3.1 ADVANTAGES OF PROPOSED SYSTEM 10

2.4 FEASIBILITY STUDY 11

2.4.1 TECHNICAL FEASIBILITY 11

2.4.2 OPERATIONAL FEASIBILITY 11

2.4.3 ECONOMIC FEASIBILITY 11

2.4.4 LEGAL FEASIBILITY 11

2.5 SOFTWARE ENGINEERING PARADIGM APPLIED 12

CHAPTER 3: SYSTEM ENVIRONMENT **13**

3.1 INTRODUCTION 14

3.2 SOFTWARE REQUIREMENTS SPECIFICATION 14

3.3 HARDWARE REQUIREMENTS SPECIFICATION 14

3.4 TOOLS, PLATFORMS 15

3.4.1 PYTHON 15

3.4.2 DJANGO 15

3.4.3 SQLITE 15

3.4.4 HTML, CSS AND JAVASCRIPT 15

3.4.5 DEVELOPMENT ENVIRONMENT 15

CHAPTER 4: SYSTEM DESIGN **16**

4.1 INTRODUCTION 17

4.2 DATA FLOW DIAGRAM 17

4.3 DATABASE DESIGN 22

4.4 ER DIAGRAM 23

4.5 UML DIAGRAMS 23

4.6 USECASE DIAGRAMS 24

ACTIVITY DIAGRAM 25
SEQUENCE DIAGRAM 26
4.7 INPUT DESIGN 27
OUTPUT DESIGN 27
4.8 TABLES 28
CHAPTER 5: SYSTEM IMPLEMENTATION 39
5.1.1 INTRODUCTION 40
5.1.2 CODING 40
CODING VALIDATION AND OPTIMIZATION 41
SAMPLE CODE 41
CHAPTER 6: SYSTEM TESTING 46
6.1 INTRODUCTION 47
BLACK BOX TESTING 47
WHITE BOX TESTING 48
6.2 UNIT TESTING 48
6.3 INTEGRATION TESTING 48
6.4 USER ACCEPTANCE TESTING 49
CHAPTER 7: SYSTEM MAINTENANCE 50
7.1 INTRODUCTION 51
7.2 MAINTENANCE 51
CORRECTIVE MAINTENANCE 51
ADAPTIVE MAINTENANCE 51
PERFECTIVE MAINTENANCE 51
PREVENTIVE MAINTENANCE 52
DATABASE MAINTENANCE 52
SECURITY MAINTENANCE 52
CHAPTER 8: FUTURE ENHANCEMENT 53
8.1 INTRODUCTION 54
8.2 MERITS 54
8.3 DEMERITS 54
8.4 FUTURE ENHANCEMENTS 54
SCREENSHOTS 56
BIBLIOGRAPHY / REFERENCES 59

ABSTRACT

The Smart Property Management and Construction Monitoring System is a Django-based web application developed to support the digital management of property listings, builder registration and approval, property discovery, customer interaction, Wishlist management, property reviews, and site-visit scheduling. The implementation found in the supplied source code is centered on real-estate property management and client engagement. Although construction monitoring appears in the project title, the current source code does not contain a construction-stage monitoring workflow; therefore, construction progress monitoring is treated as a future enhancement rather than an implemented module.

The application follows Django's Model-Template-View approach and uses SQLite as the configured database engine. The project contains separate application components for administration, builders, and users, together with the main villa project module. Administrators can review builder requests, approve or remove builders, view client queries, inspect scheduled visits, and access a dashboard containing counts for users, builders, pending builders, properties, bookings, and queries. Approved builders can add, edit, and delete property listings, while registered users can browse and filter properties, maintain a wishlist, submit reviews, and schedule property visits.

The supplied test modules contain Django TestCase imports but no defined test methods. In addition, an execution attempt in the documentation-generation environment could not run the Django test suite because Django was not installed in that environment. Accordingly, this report describes the available testing structure and proposes black-box and white-box test cases without claiming that the project's complete test suite has passed.

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

The real-estate domain requires accurate property information, accessible property listings, reliable communication between customers and property providers, and organized management of site-visit requests. Managing these activities manually can be time-consuming and may result in inconsistent information and difficulties in coordinating customer requirements. A centralized web-based application can help simplify these processes by maintaining property information and customer records in a common system.

The Smart Property Management and Construction Monitoring System is a web-based application developed using Python and the Django framework. It provides a platform for property listing management, builder registration and approval, property browsing, customer interaction, Wishlist management, property reviews, and site-visit scheduling. The application uses SQLite as its database and Django templates to provide the user interface.

The system provides separate functions for administrators, builders, and registered users. Administrators can review builder registration requests, manage builder records, view customer enquiries, inspect scheduled visits, and access dashboard information. Approved builders can manage property listings, while registered users can browse properties, apply available search filters, maintain a Wishlist, submit reviews, and schedule property visits.

The main purpose of the system is to organize property-related information and improve interaction between customers, builders, and administrators through a common web application.

1.2 BACKGROUND

The traditional process of searching for and managing properties often depends on advertisements, telephone calls, individual communication, and manually maintained records. Property details may be distributed across different sources, making it difficult for customers to compare available properties and obtain consistent information. Builders may also face difficulties in organizing property details and responding to customer enquiries. A centralized property management application can reduce these difficulties by storing property information in a database and making it accessible through a web interface. It can also provide a structured process for builder registration, administrative approval, property listing management, and customer site-visit requests.

The Smart Property Management and Construction Monitoring System addresses these requirements through a Django-based application. Builders can register and submit property information after obtaining the required approval. Customers can browse properties and use

available filters to identify properties that match their requirements. Registered users can also save preferred properties, submit reviews, and request site visits.

The system therefore provides a common platform for managing property information and coordinating important interactions between the people involved in property transactions.

1.3 PROBLEM STATEMENT

The project addresses the problem of managing property listings and customer interactions through a single web application. Without a centralized system, property information may be fragmented, builder verification may be difficult to coordinate, and requests for visits or enquiries may have to be handled separately. The application seeks to provide a common database-backed environment for listing management, builder approval, customer interaction, and booking records.

1.4 PROJECT SCOPE AND RELEVANCE

The implemented scope covers property listing creation and maintenance by approved builders, property browsing and filtering by users, user registration and login, builder registration and administrative approval, wishlist operations, property reviews, contact enquiries, and site-visit scheduling. The scope also includes an administrator dashboard that aggregates record counts and recent information.

- Management of property records with property photographs and descriptive attributes.
- Registration of users and builders with validation and session-based login.
- Administrative review and approval of builder applications.
- Property discovery using category, keyword, price range, and bedroom filters.
- Wishlist addition and removal for authenticated users.
- Property rating and review submission by authenticated users.
- Site-visit scheduling and booking display for users, builders, and administrators.
- Storage and display of customer contact queries.

The title includes construction monitoring, but no construction-stage, progress, milestone, labour, material, or site-monitoring model is present in the supplied source code. Such functionality is therefore outside the implemented scope.

1.5 OBJECTIVES OF THE PROPOSED SYSTEM

1. To provide a centralized web platform for maintaining and browsing property listings.
2. To allow builders to register and require administrative approval before property management, access is granted.
3. To enable administrators to manage builder requests, approved builders, queries, and scheduled visits.
4. To allow registered users to search and filter property listings using available property attributes.
5. To provide wishlist and property-review functionality for authenticated users.
6. To support site-visit scheduling and display of booking information.
7. To validate important registration inputs such as email, phone number, password, and password confirmation.
8. To use Django sessions to restrict selected operations to authenticated roles.

1.6 EXISTING SYSTEM

In a conventional property-management workflow, listing details may be maintained through separate documents, spreadsheets, advertisements, or independent communication channels. Builder verification and customer appointment requests can require manual coordination. Such a workflow does not provide a single, consistent interface for maintaining property information and recording customer interactions.

1.7 PROPOSED SYSTEM

The proposed system is a database-backed Django web application that brings property listing management and customer interaction into one environment. It maintains user, builder, approved-builder, property, booking, wishlist, review, and query records using Django models and SQLite. Role-sensitive views use session values to determine whether an administrator, builder, or registered user may access particular operations.

1.8 ADVANTAGES

- Centralized storage of property and interaction records.
- Administrative approval of builder registrations before builder access.
- Search and filtering of available properties.

- Image support for user, builder, and property records.
- Wishlist and review facilities for registered users.
- Site-visit booking records that can be viewed by relevant roles.
- Dashboard summary information for administrators.

CHAPTER 2
SYSTEM ANALYSIS

2.1 INTRODUCTION

System analysis examines the problem domain, existing practices, proposed solution, requirements, and feasibility of the project. For this system, the analysis is based on the actual Django applications, models, URL patterns, templates, session checks, and database configuration found in the supplied project.

2.2 EXISTING SYSTEM

The existing approach can be represented by decentralized property information and customer communication. Property information may be distributed among sellers or builders, while customer interest is handled through individual conversations. Builder registration and approval are also difficult to coordinate without a dedicated workflow.

2.2.1 LIMITATIONS OF EXISTING SYSTEM

- Property information may be scattered across different sources.
- Builder approval and verification can require manual coordination.
- Customer site-visit requests may not have a centralized record.
- There is limited support for maintaining user-specific favourites.
- Customer feedback may not be associated with a property record in a structured way.
- Administrative oversight of listings and requests is difficult when records are maintained separately.

2.3 PROPOSED SYSTEM

The proposed system stores the major business records in a Django database and exposes them through role-aware web views. A guest can browse the public interface and submit a contact query. A user can register, log in, browse and filter properties, add or remove wishlist items, submit a review, and schedule a visit. A builder can register, wait for administrative approval, and then manage property listings. An administrator can review builder requests, approve or delete builders, view queries, view bookings, and use the dashboard.

2.3.1 ADVANTAGES OF PROPOSED SYSTEM

- Reduces manual record handling for property listings and visit requests.
- Provides a common interface for users, builders, and administrators.
- Supports builder approval before builder property management.

- Improves property discovery through filtering.
- Maintains user-specific wishlist records.
- Stores customer reviews and site-visit bookings in the database.

2.4 FEASIBILITY STUDY

Feasibility analysis determines whether the proposed system can be developed and used with the available technical, operational, economic, and legal resources. The supplied project is a Django web application using SQLite, HTML templates, CSS, JavaScript, and static assets. These technologies are suitable for local development and academic demonstration.

2.4.1 TECHNICAL FEASIBILITY

The project is technically feasible because it uses Python with Django 5.2.5, a built-in SQLite database configuration, standard Django sessions, template rendering, and static/media file handling. The application is organized into the main villa project and separate admin_app, user_app, and builder_app components. The supplied archive also contains a local virtual environment, although the generation environment used for this report did not use that environment to execute Django.

2.4.2 OPERATIONAL FEASIBILITY

The system is operationally feasible for a web-based property-management workflow. Users interact through forms and property cards, builders use dedicated listing screens, and administrators receive dedicated management pages. The interface is based on HTML templates and common web controls, reducing the learning effort required for basic operations.

2.4.3 ECONOMIC FEASIBILITY

The configured application uses open-source technologies such as Python, Django, SQLite, HTML, CSS, and JavaScript. No paid database server or proprietary framework is required by the source code. Therefore, the software stack is suitable for an academic project and low-cost deployment.

2.4.4 LEGAL FEASIBILITY

The project uses common open-source development technologies and includes static third-party front-end assets. A production deployment should separately review the licences and usage terms of all external themes, libraries, fonts, and images included with or referenced by the project.

User-provided personal data such as names, emails, phone numbers, photographs, and passwords also require appropriate privacy and security handling in a real deployment.

2.5 SOFTWARE ENGINEERING PARADIGM APPLIED

The implementation follows Django's Model-Template-View (MTV) pattern. Models define persistent data structures, views contain request-processing logic, and templates render the user interface. URL configuration maps browser requests to the corresponding views. Session values provide a lightweight role distinction for administrators, users, and builders. The code is organized into multiple Django applications, which provides functional separation between administration, property management, and user interaction.

CHAPTER 3
SYSTEM ENVIORNMENT

3.1 INTRODUCTION

The system environment specifies the software and hardware resources required to develop and run the application. The supplied project configuration identifies Django 5.2.5, SQLite, static and media directories, and the Django session middleware. The front end is implemented using Django templates with HTML, CSS, JavaScript, and external interface libraries.

3.2 SOFTWARE REQUIREMENTS SPECIFICATION

Component	Specification
Operating System	Microsoft Windows compatible development environment
Programming Language	Python
Framework	Django 5.2.5
Database	SQLite
Web Interface	HTML, CSS and JavaScript with Django Templates
Session Management	Django Session Middleware
Static Files	Django static files configuration
Media Files	Django MEDIA_ROOT / MEDIA_URL configuration
Development Tool	Visual Studio Code or another Python-compatible IDE
Browser	Modern web browser

3.3 HARDWARE REQUIREMENTS SPECIFICATION

The source code does not prescribe a particular processor, monitor, keyboard, or hard-disk model. For academic development, a conventional personal computer capable of running Python, Django, SQLite, a modern browser, and an IDE is sufficient.

Resource	Recommended Academic Configuration
Processor	Modern dual-core or better processor
RAM	4 GB minimum; 8 GB or more recommended
Storage	At least 10 GB free space for development tools and project files
Display	Standard 1366×768 or higher display
Network	Internet connection when external CDN assets are required

3.4 TOOLS, PLATFORMS

3.4.1 PYTHON

Python is the primary programming language of the application. The source code uses Python modules for Django models, views, URL configuration, regular-expression validation, request handling, database operations through the ORM, and session-based access control.

3.4.2 DJANGO

Django is the web framework used by the project. The settings file identifies Django 5.2.5 as the generated framework version. The project uses Django models, template rendering, URL routing, session middleware, CSRF middleware, authentication middleware, static-file handling, and media-file configuration.

3.4.3 SQLITE

SQLite is the configured database engine. The settings file defines the database as a file named db.sqlite3 under the project base directory. The database schema represented in the supplied database contains the project tables as well as Django framework tables.

3.4.4 HTML, CSS AND JAVASCRIPT

HTML templates define the page structure, forms, navigation, property cards, dashboard views, and management pages. CSS controls the visual presentation, including the Villa-style real-estate theme. JavaScript in the templates supports client-side interface behaviour such as the mortgage/EMI calculator, property comparison interface, and interactive controls. These client-side tools should be distinguished from database-backed server features.

3.4.5 DEVELOPMENT ENVIRONMENT

The archive contains a Django project with manage.py, a project package named villa, three application packages, templates, static assets, media files, database files, and a local virtual environment. The source includes both the active project tree and an additional nested copy under a villa directory; the active configuration used for this report is the top-level project tree because its manage.py points to villa. Settings and settings define the installed applications.

CHAPTER 4
SYSTEM DESIGN

4.1 INTRODUCTION

System design translates user requirements and conceptual designs into technical specifications and processing flows. After identifying user needs and gathering relevant information, a new system is proposed, including various tables, maintenance, and report generation. The design emphasizes efficiency and user-friendliness while ensuring all proposed functions are met. Key steps involve creating user-acceptable input and output forms, understanding the system through data flow diagrams, and applying normalization in database design. Program specifications detail components and processes, while validation rules ensure outputs meet standards and security checks to protect data from unauthorized access. Overall, input and output designs follow predefined guidelines for effective navigation, converting user inputs into computer formats and generating requested information in hard or soft copies.

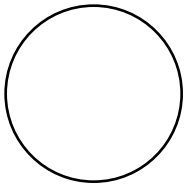
4.2 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) illustrates the flow of data and processes within a system, serving as a structured analysis and design tool. It helps analysts specify user needs and determine data flow into and out of the system using a set of symbols that don't imply physical implementation. DFDs review existing systems, prepare input and output specifications, and outline implementation plans. Context flow diagrams, data flow diagrams, and flowcharts are crucial for successful system design. Efficiently designing these diagrams minimizes flaws and ensures operational longevity. DFDs use four basic symbols to represent data sources, flows, transformations, and storage, with nodes indicating where data transformations occur.

Main symbols used in the data flow diagram are:



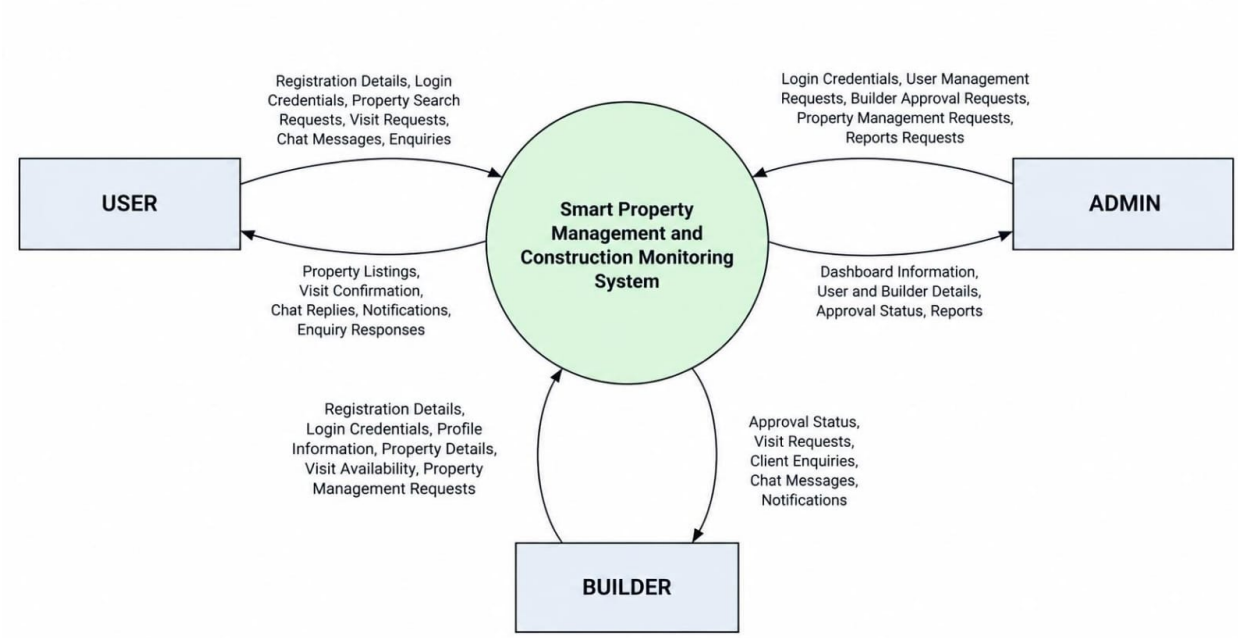
A rectangle represents a data source



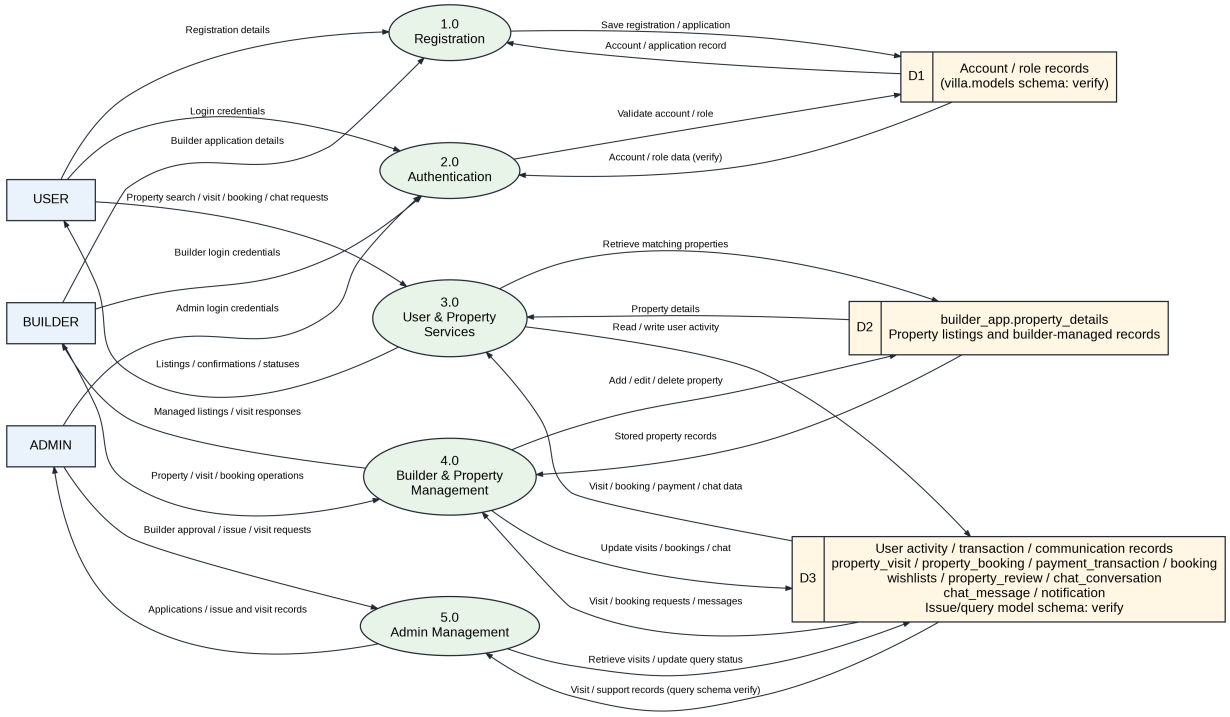
An enclosed figure represents a process that transform the data stream.

A directed line represents the flow of data.

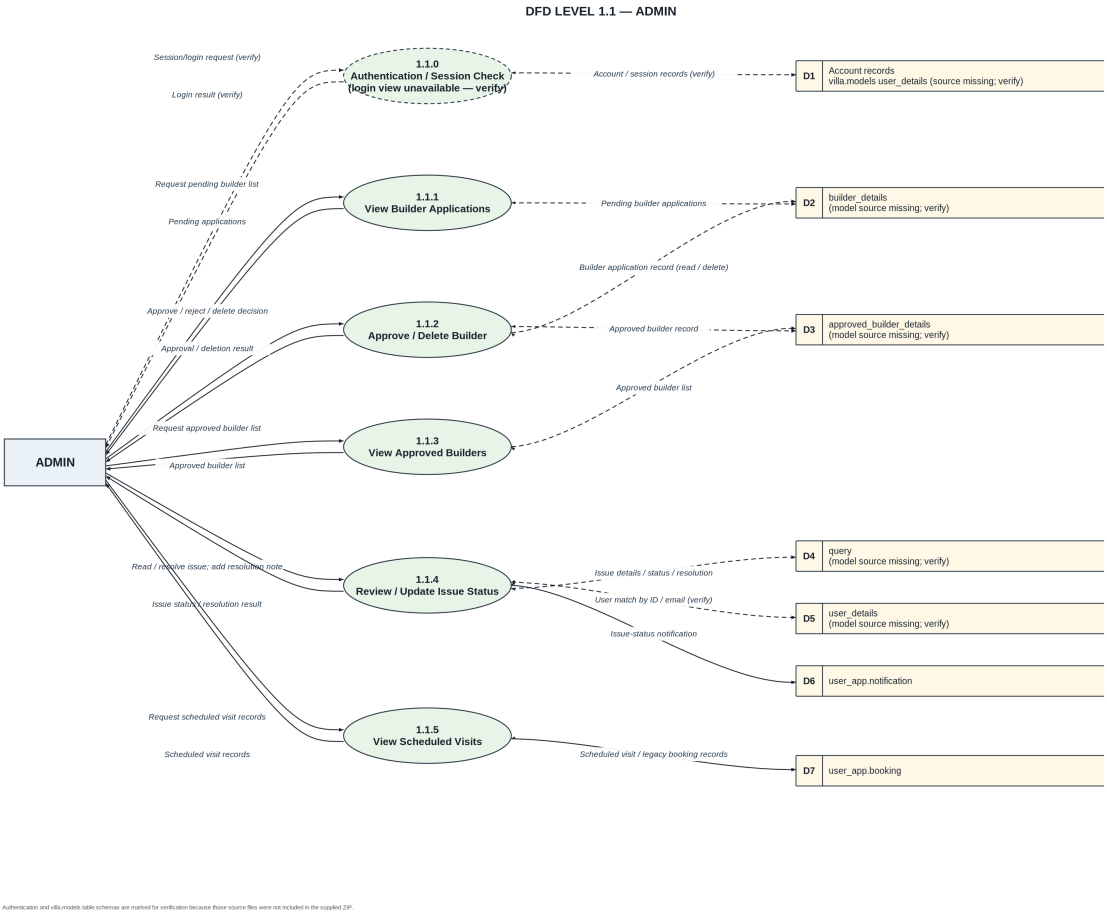
LEVEL 0 – CONTEXT LEVEL



LEVEL 1

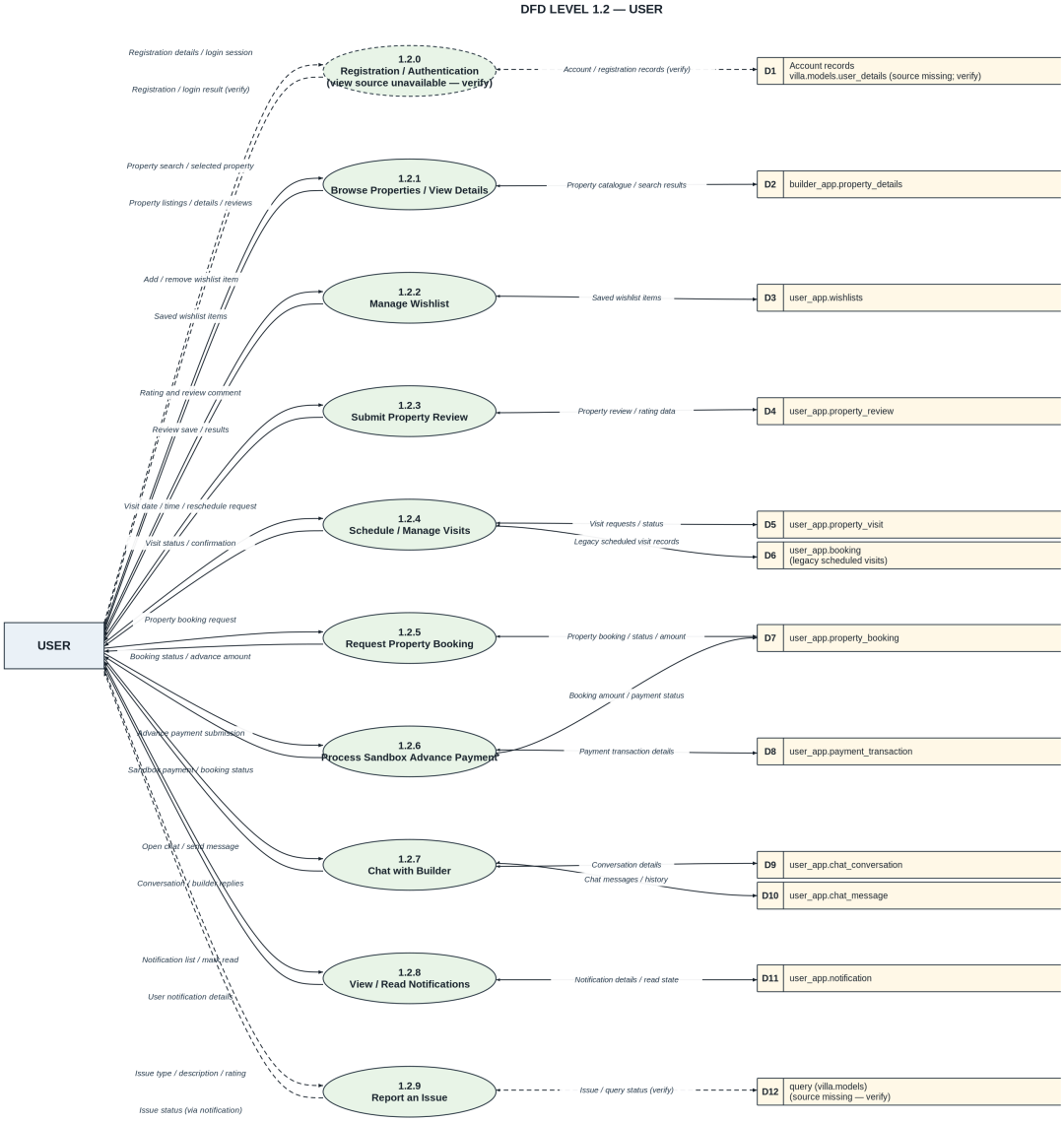


LEVEL 1.1 – ADMIN



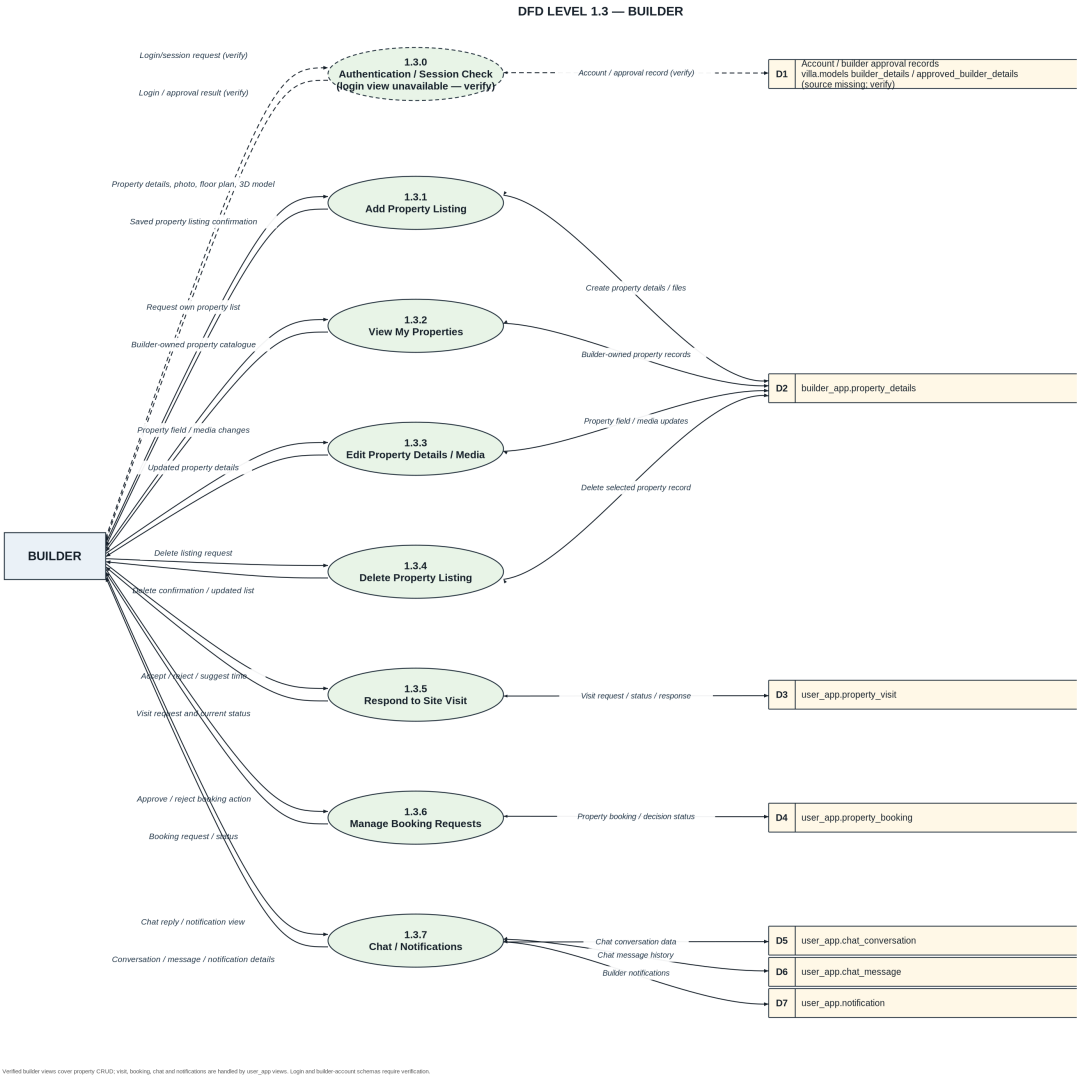
Authentication and villa.models table schemas are marked for verification because those source files were not included in the supplied ZIP.

LEVEL 1.2– USER



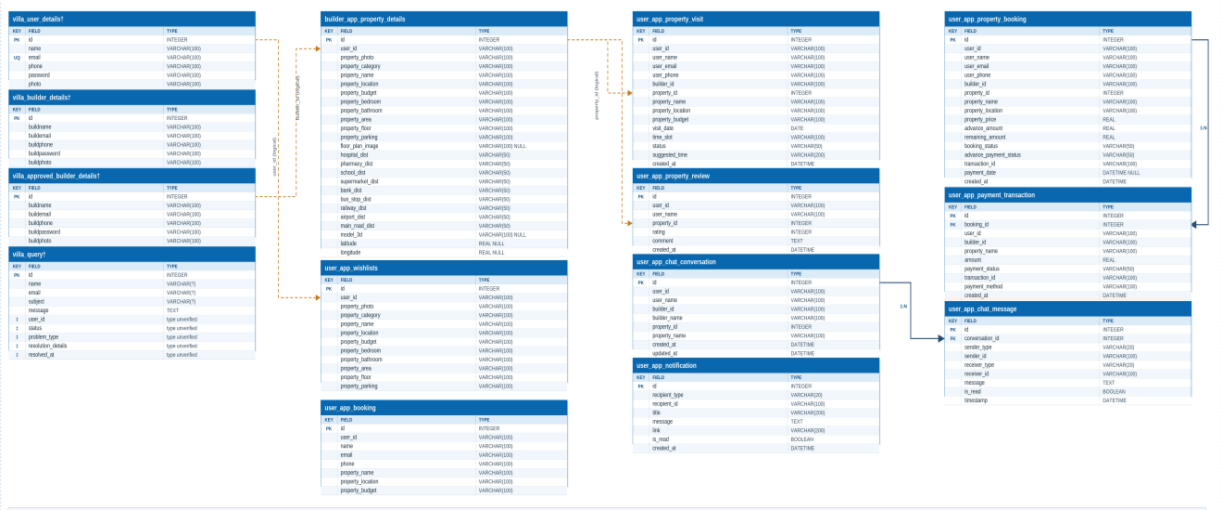
User data models are verified in user_app/models.py except query and account models imported from missing villa.models; those are marked for verification.

LEVEL 1.3 – BUILDER



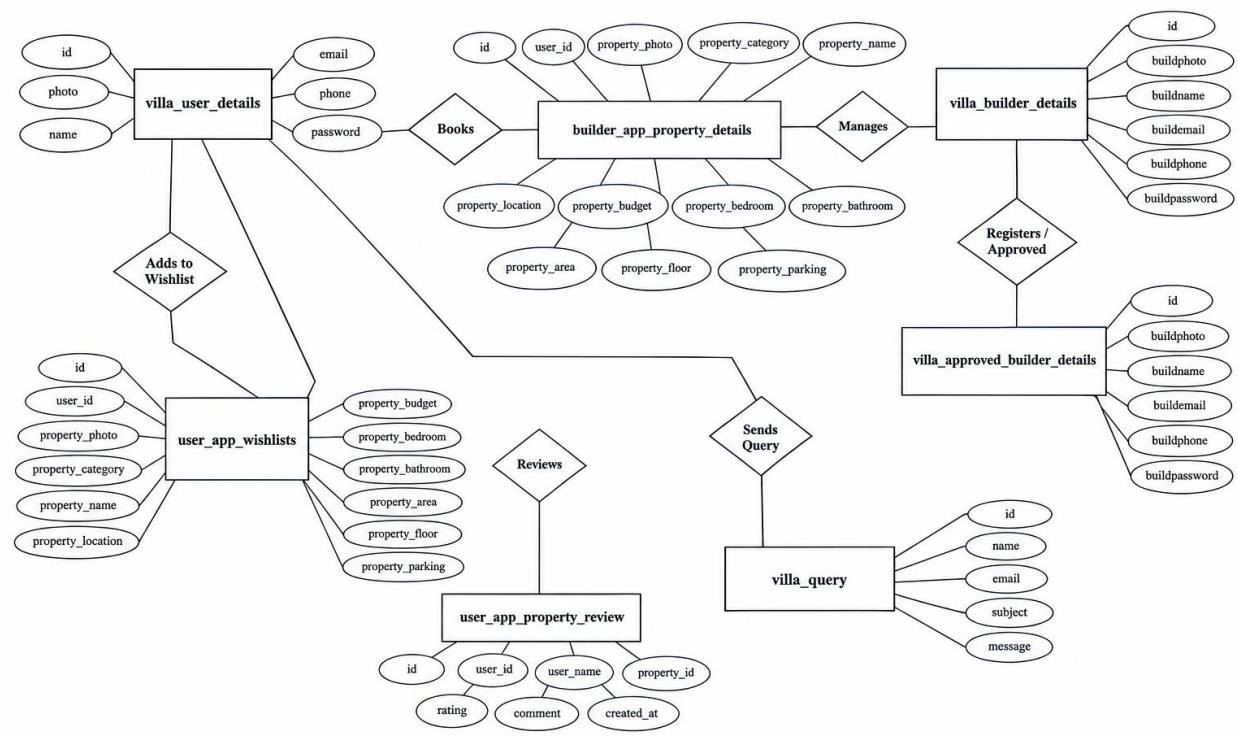
Verified builder views cover property CRUD; visit, booking, chat and notifications are handled by user_app views. Login and builder-account schemas require verification.

4.3 DATABASE DESIGN



4.4 ER DIAGRAM

The Entity-Relationship Diagram (ERD) is a graphical representation used in database design to illustrate the logical structure of a database system. It visually depicts the entities (objects), attributes (properties), relationships, and cardinalities (how entities are connected) within a database. ER diagrams are a fundamental tool for database designers and developers to conceptualize, communicate, and plan the structure of a database before its actual implementation. Entities: Represent real-world objects or concepts, such as customers, products, employees, etc Attributes: Describe the properties or characteristics of entities, like names, dates, quantities, etc. Relationships: Depict how entities are related to each other, indicating associations, dependencies, or interactions. Cardinality: Defines the number of instances of one entity that can be associated with another entity. Common cardinalities include one-to-one, one to-many, and many-to-many.



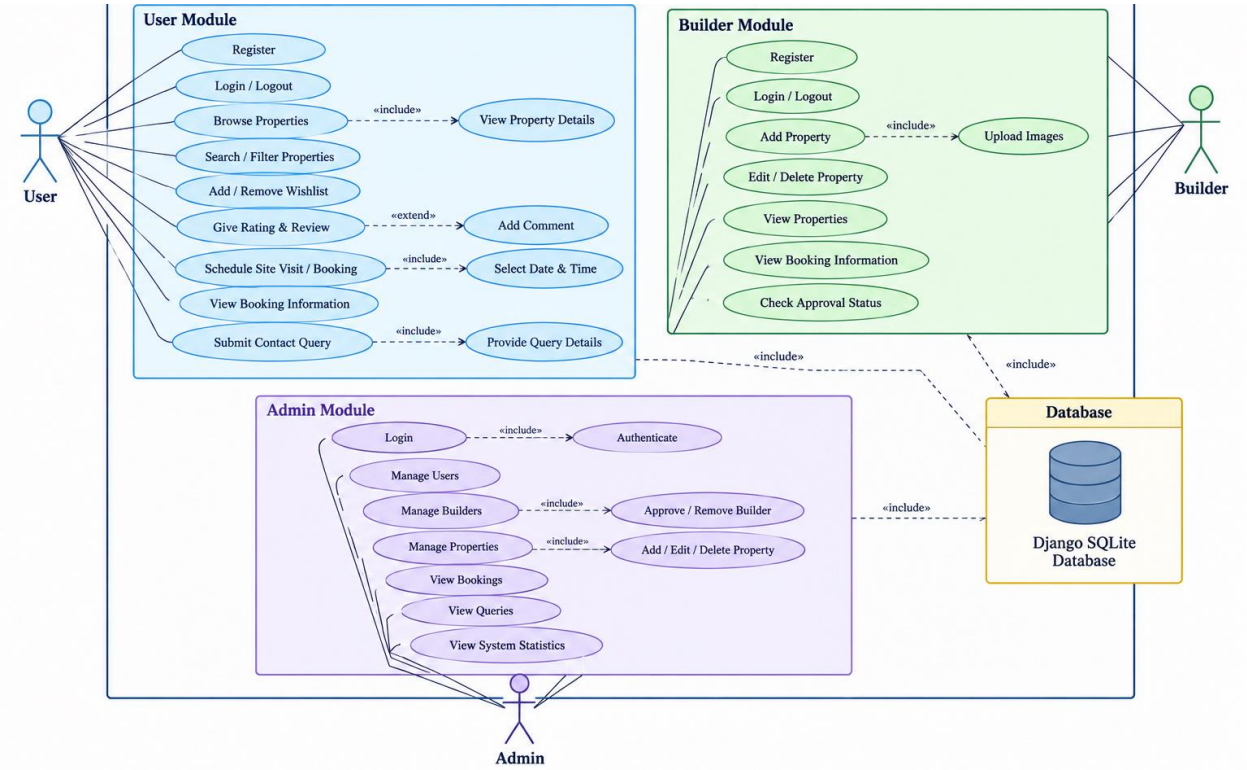
4.5 UML DIAGRAMS

A sequence diagram is a dynamic and visual representation in software engineering, illustrating the interaction between objects or components within a system over time. It focuses on the chronological sequence of messages exchanged between these objects to achieve specific functionality, making it particularly useful for understanding the behaviour and flow of a system during runtime. In a sequence diagram, objects (also known as lifelines) are represented as

vertical lines, and the interactions between them are depicted using arrows showing the messages passed back and forth. These interactions can include method calls, signals, and other forms of communication. The horizontal axis represents time, allowing the diagram to capture the chronological order of these interactions.

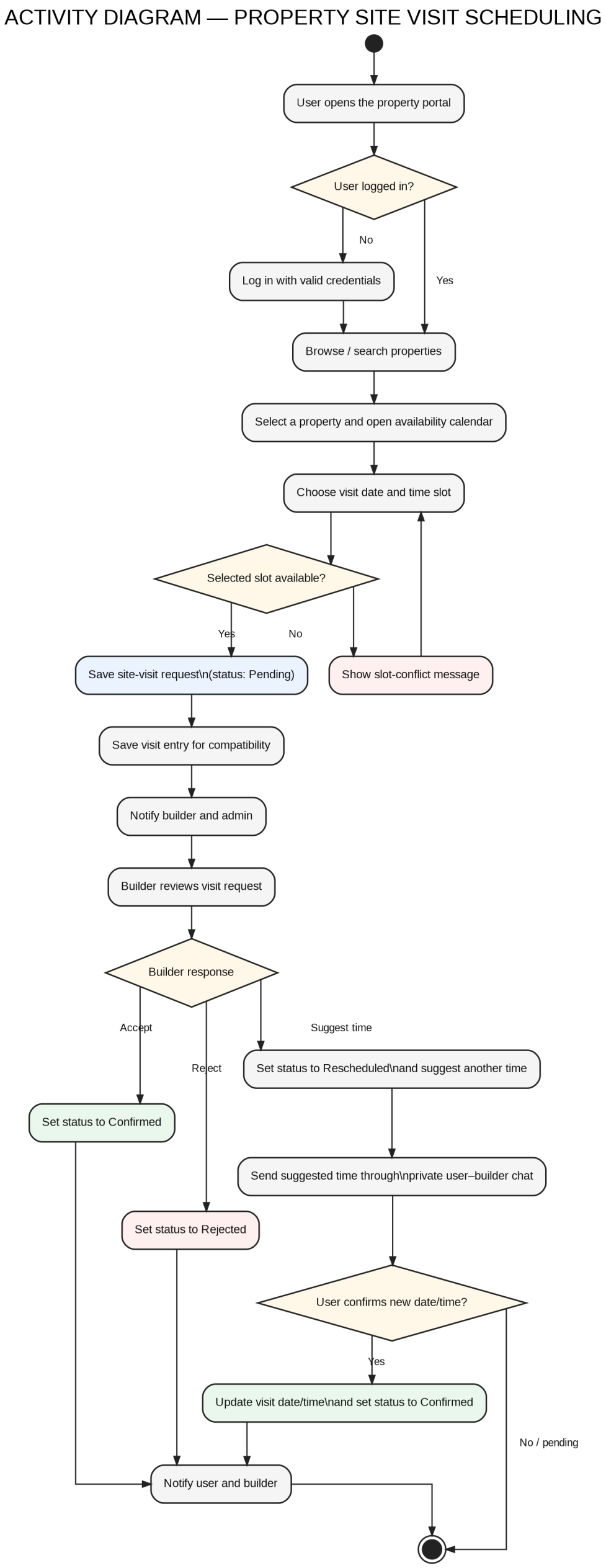
4.6 USECASE DIAGRAMS

Use cases help to determine the functionality and features of the software from user's perspective. A use case describes how a user interacts with the system by defining the steps required to accomplish a specific goal. Variations in the sequence of steps describe various 27 scenarios. In the diagram the stick figure represents an actor that is associated with one category of user. In the use-case diagram the use cases are displayed as ovals. A use case diagram displays the relationship among actors and use cases. The two main components of a use case diagram are use cases and actors. The actors are connected by lines to the use cases that they carry out. The use cases are placed in a rectangle but the actors are not. This rectangle is a visual remainder of the system boundaries and that the actors are outside the system.



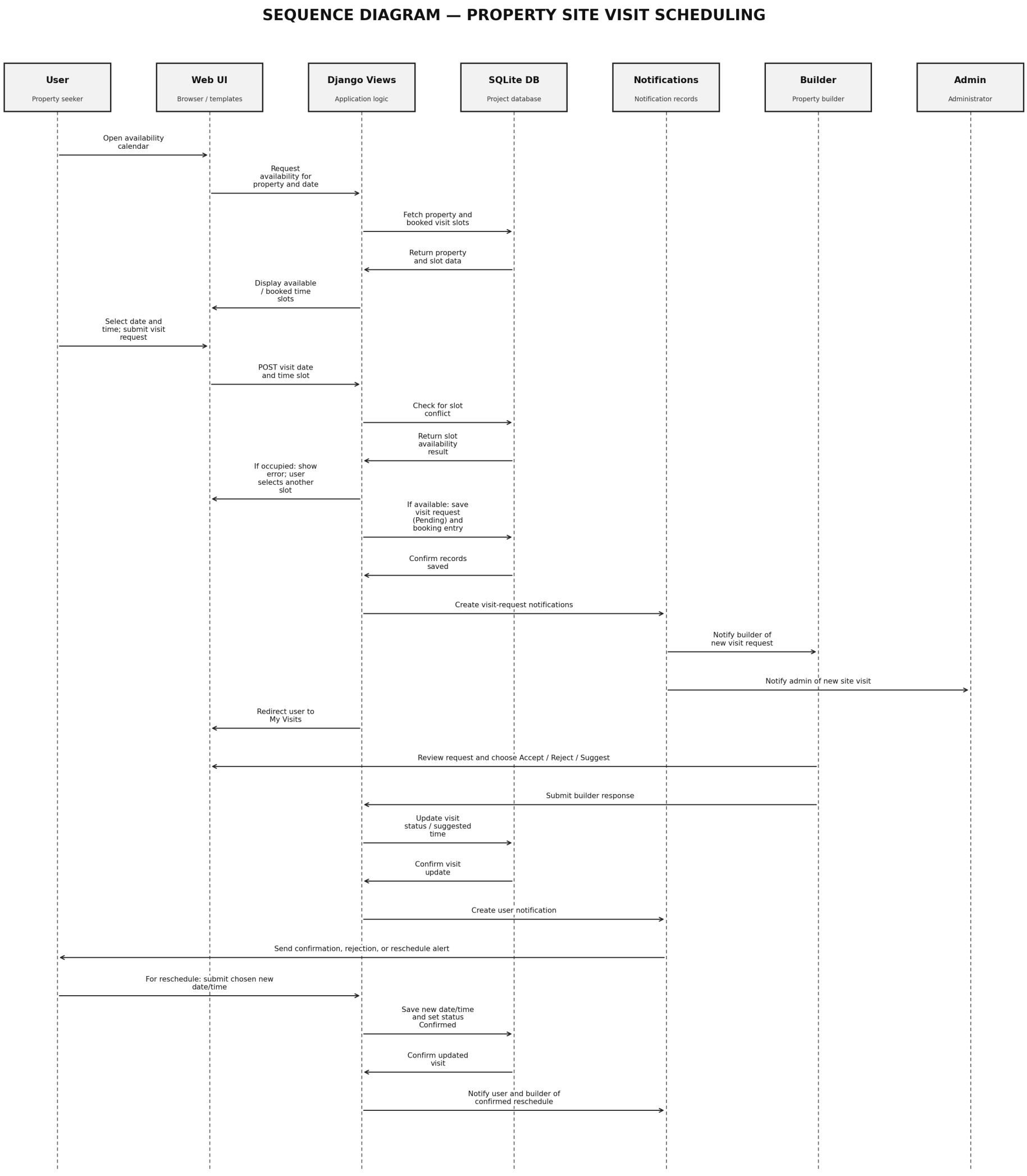
Activity Diagram

An activity diagram shows the actions and decision points in a workflow and the order in which they occur. In this system, it follows a user's site-visit request from opening a property's availability calendar through slot checking and builder review. It also shows acceptance, rejection, and suggested-time paths, including the user's confirmation of a proposed new visit time.



Sequence Diagram

A sequence diagram shows how actors and system components exchange messages over time. For this site-visit workflow, it follows the user's availability request and visit submission through the Web UI and Django views to the SQLite database. It then shows notifications to the builder and administrator, the builder's decision, and the status and notification updates, including a user-confirmed reschedule.



4.7 INPUT DESIGN

The input is the set of values that is provided by the user to the system. The input x processing. The input design is the process of converting the user-oriented inputs into computer-based formats. The data fed into the system using simple interactive forms. The forms have been supplied with messages so that user can enter data without facing any difficulty. The data is validated wherever it requires in the project. The input data have to be validated, edited, organized, and accepted by the system before being proposed to produce the outputs.

The main objectives of input design are follows:

- Produce effective method of input
- Achieve high level accuracy
- Ensure that the input is acceptable and understood by the user

The different types of input data handled by the system are:

EXTERNAL

They are the primary inputs to the system. The external input is what the user supplies to the system. The user can give different types of external inputs in this project such as add new threads, post reply. When the external inputs are obtained from the user, these inputs are transferred to the system as messages. These messages are captured and handled as input for further processing. In this project the input design is done with PHP codes. The external inputs are data given to the system by the user such as username and password for authentication process. The external input also includes the request as per the user’s interest for displaying today’s, yesterdays and last week’s threads/posts and its replies. The internal input covers the fetching of data from the database and it will be the input for displaying the results of the screen.

INTERNAL

The internal inputs required by the system are provided through the Graphical User Interface (GUI). This GUI design enhances user experience by minimizing input errors and ensuring clarity during data entry. By guiding users through the input process, the system reduces the likelihood of incorrect conclusions and improves overall accuracy.A quality output meets the end user's requirements and presents information clearly. In any system, the results of processing are communicated to users and other systems through output design, which determines how information is displayed to address immediate needs. Effective output design is crucial as it serves as the primary source of information for users, enhancing the relationship between the system and the user while aiding in decision-making.The objectives of output design include

conveying information about past activities, current status, and highlighting significant events. Outputs refer to the results and information generated by the system, primarily aimed at communicating processing outcomes to users. For each query submitted by the user, the system displays the corresponding output. It's essential to test the output for each query to ensure the accuracy and reliability of the results.

4.8 TABLES

villa_user_details

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique user record ID
photo	VARCHAR(100)	NOT NULL	Stored path to the user's profile photo
name	VARCHAR(100)	NOT NULL	User's name
email	VARCHAR(100)	NOT NULL	User's email address
phone	VARCHAR(100)	NOT NULL	User's phone number
password	VARCHAR(100)	NOT NULL	User's account password value

villa_builder_details

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique builder application ID
buildphoto	VARCHAR (100)	NOT NULL	Stored path to the builder's photo
buildname	VARCHAR (100)	NOT NULL	Builder's name
buildemail	VARCHAR (100)	NOT NULL	Builder's email address
buildphone	VARCHAR (100)	NOT NULL	Builder's phone number
buildpassword	VARCHAR (100)	NOT NULL	Builder's account password value

villa_builder_details

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique approved-builder ID
buildphoto	VARCHAR(100)	NOT NULL	Stored path to the builder's photo
buildname	VARCHAR(100)	NOT NULL	Approved builder's name
buildemail	VARCHAR(100)	NOT NULL	Approved builder's email address
buildphone	VARCHAR(100)	NOT NULL	Approved builder's phone number
buildpassword	VARCHAR(100)	NOT NULL	Builder's account

builder_app_property_details

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique property ID
user_id	VARCHAR(100)	NOT NULL	ID of the builder who listed the property; not a database-enforced foreign key
property_photo	VARCHAR(100)	NOT NULL	Stored path to the property photo
property_category	VARCHAR(100)	NOT NULL	Property category
property_name	VARCHAR(100)	NOT NULL	Property name
property_location	VARCHAR(100)	NOT NULL	Property location
property_budget	VARCHAR(100)	NOT NULL	Property budget or listed price
property_bedroom	VARCHAR(100)	NOT NULL	Number of bedrooms
property_bathroom	VARCHAR(100)	NOT NULL	Number of bathrooms
property_area	VARCHAR(100)	NOT NULL	Property area
property_floor	VARCHAR(100)	NOT NULL	Property floor information
property_parking	VARCHAR(100)	NOT NULL	Property parking information
airport_dist	VARCHAR(50)	NOT NULL; Model default: 22 km	Distance to the airport
bank_dist	VARCHAR(50)	NOT NULL; Model default: 1.0 km	Distance to a bank
bus_stop_dist	VARCHAR(50)	NOT NULL; Model default: 0.4 km	Distance to a bus stop

Field	Data type	Constraint	Description
floor_plan_image	VARCHAR(100)	NULL	Stored path to an optional floor-plan image
hospital_dist	VARCHAR(50)	NOT NULL; Model default: 1.5 km	Distance to a hospital
latitude	REAL	NULL	Property latitude
longitude	REAL	NULL	Property longitude
main_road_dist	VARCHAR(50)	NOT NULL; Model default: 0.3 km	Distance to a main road
pharmacy_dist	VARCHAR(50)	NOT NULL; Model default: 0.5 km	Distance to a pharmacy
railway_dist	VARCHAR(50)	NOT NULL; Model default: 3.5 km	Distance to a railway station
school_dist	VARCHAR(50)	NOT NULL; Model default: 2.0 km	Distance to a school
supermarket_dist	VARCHAR(50)	NOT NULL; Model default: 1.2 km	Distance to a supermarket
model_3d	VARCHAR(100)	NULL	Stored path to an optional 3D model file

user_app_booking

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique visit-booking record ID
user_id	VARCHAR(100)	NOT NULL	ID of the user who requested the visit
name	VARCHAR(100)	NOT NULL	User’s name
email	VARCHAR(100)	NOT NULL	User’s email address
phone	VARCHAR(100)	NOT NULL	User’s phone number
property_name	VARCHAR(100)	NOT NULL	Name of the property to visit
property_location	VARCHAR(100)	NOT NULL	Location of the property
property_budget	VARCHAR(100)	NOT NULL	Property budget or listed price

user_app_wishlists

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique wishlist record ID
user_id	VARCHAR(100)	NOT NULL	ID of the user who saved the property
property_photo	VARCHAR(100)	NOT NULL	Stored path to the property photo
property_category	VARCHAR(100)	NOT NULL	Property category
property_name	VARCHAR(100)	NOT NULL	Property name
property_location	VARCHAR(100)	NOT NULL	Property location
property_budget	VARCHAR(100)	NOT NULL	Property budget or listed price
property_bedroom	VARCHAR(100)	NOT NULL	Number of bedrooms
property_bathroom	VARCHAR(100)	NOT NULL	Number of bathrooms
property_area	VARCHAR(100)	NOT NULL	Property area
property_floor	VARCHAR(100)	NOT NULL	Property floor information
property_parking	VARCHAR(100)	NOT NULL	Property parking information

user_app_property_review

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique property-review ID
user_id	VARCHAR(100)	NOT NULL	ID of the reviewing user
user_name	VARCHAR(100)	NOT NULL	Name of the reviewing user
property_id	INTEGER	NOT NULL	ID of the reviewed property; not a database-enforced foreign key
rating	INTEGER	NOT NULL; Model default: 5	Review rating
comment	TEXT	NOT NULL	Review comment
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the review was created

Table: user_app_property_visit

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique property-visit request ID
user_id	VARCHAR(100)	NOT NULL	ID of the user requesting the visit
user_name	VARCHAR(100)	NOT NULL	User's name
user_email	VARCHAR(100)	NOT NULL	User's email address
user_phone	VARCHAR(100)	NOT NULL	User's phone number
builder_id	VARCHAR(100)	NOT NULL	ID of the property's builder
property_id	INTEGER	NOT NULL; Model default: 0	ID of the property; not a database-enforced foreign key
property_name	VARCHAR(100)	NOT NULL	Property name
property_location	VARCHAR(100)	NOT NULL	Property location
property_budget	VARCHAR(100)	NOT NULL	Property budget or listed price
visit_date	DATE	NOT NULL	Requested visit date
time_slot	VARCHAR(100)	NOT NULL	Requested visit time slot
status	VARCHAR(50)	NOT NULL; Model default: Pending	Visit request status
suggested_time	VARCHAR(200)	NOT NULL; Model default: "	Alternative time suggested for the visit
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the request was created

Table: user_app_chat_conversation

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique conversation ID
user_id	VARCHAR(100)	NOT NULL	ID of the user in the conversation
user_name	VARCHAR(100)	NOT NULL; Model default: "	User's name
builder_id	VARCHAR(100)	NOT NULL	ID of the builder in the conversation
builder_name	VARCHAR(100)	NOT NULL; Model default: "	Builder's name
property_id	INTEGER	NOT NULL; Model default: 0	Related property ID; not a database-enforced foreign key

Field	Data type	Constraint	Description
property_name	VARCHAR(100)	NOT NULL; Model default: ''	Related property name
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the conversation was created
updated_at	DATETIME	NOT NULL; updated automatically by Django	Date and time the conversation was last updated

Table: user_app_chat_message

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique message ID
sender_type	VARCHAR(20)	NOT NULL	Type of the message sender
sender_id	VARCHAR(100)	NOT NULL	ID of the message sender
receiver_type	VARCHAR(20)	NOT NULL	Type of the message recipient
receiver_id	VARCHAR(100)	NOT NULL	ID of the message recipient
message	TEXT	NOT NULL	Message content
is_read	BOOLEAN	NOT NULL; Model default: False	Whether the message has been read
timestamp	DATETIME	NOT NULL; set automatically by Django	Date and time the message was sent
conversation_id	BIGINT	NOT NULL; FK → user_app_chat_conversation.id; CASCADE on delete	Conversation containing the message

Table: user_app_notification

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique notification ID
recipient_type	VARCHAR(20)	NOT NULL	Type of notification recipient
recipient_id	VARCHAR(100)	NOT NULL	ID of the notification recipient
title	VARCHAR(200)	NOT NULL	Notification title
message	TEXT	NOT NULL	Notification content
link	VARCHAR(200)	NOT NULL; Model default: ''	Optional link associated with the notification
is_read	BOOLEAN	NOT NULL; Model default: False	Whether the notification has been read

Field	Data type	Constraint	Description
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the notification was created

Table: user_app_property_booking

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique property-booking ID
user_id	VARCHAR(100)	NOT NULL	ID of the user making the booking
user_name	VARCHAR(100)	NOT NULL	User’s name
user_email	VARCHAR(100)	NOT NULL	User’s email address
user_phone	VARCHAR(100)	NOT NULL	User’s phone number
builder_id	VARCHAR(100)	NOT NULL	ID of the property’s builder
property_id	INTEGER	NOT NULL	ID of the booked property; not a database-enforced foreign key
property_name	VARCHAR(100)	NOT NULL	Property name
property_location	VARCHAR(100)	NOT NULL	Property location
property_price	REAL	NOT NULL; Model default: 0.0	Property price
advance_amount	REAL	NOT NULL; Model default: 0.0	Advance payment amount
remaining_amount	REAL	NOT NULL; Model default: 0.0	Remaining amount due
booking_status	VARCHAR(50)	NOT NULL; Model default: Requested	Booking status
advance_payment_status	VARCHAR(50)	NOT NULL; Model default: Pending	Advance-payment status
transaction_id	VARCHAR(100)	NOT NULL; Model default: "	Payment transaction ID, if available
payment_date	DATETIME	NULL	Date and time of payment
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the booking was created

Table: user_app_payment_transaction

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique payment transaction record ID
user_id	VARCHAR(100)	NOT NULL	ID of the user making the payment
builder_id	VARCHAR(100)	NOT NULL	ID of the property's builder
property_name	VARCHAR(100)	NOT NULL	Name of the property associated with the payment
amount	REAL	NOT NULL	Payment amount
payment_status	VARCHAR(50)	NOT NULL; Model default: Successful	Payment status
transaction_id	VARCHAR(100)	NOT NULL	Transaction identifier
payment_method	VARCHAR(100)	NOT NULL; Model default: Test Sandbox Gateway	Payment method
created_at	DATETIME	NOT NULL; set automatically by Django	Date and time the transaction was created
booking_id	BIGINT	NOT NULL; FK → user_app_property_booking.id; CASCADE on delete	Booking associated with the payment

Django Framework Tables

These tables are created and maintained by Django for authentication, permissions, admin history, sessions, and migration tracking.

Table: auth_user

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique Django user ID
password	VARCHAR(128)	NOT NULL	Django authentication password field
last_login	DATETIME	NULL	Date and time of the user's last login
is_superuser	BOOLEAN	NOT NULL	Whether the user has all permissions
username	VARCHAR(150)	NOT NULL, UNIQUE	Unique login username
last_name	VARCHAR(150)	NOT NULL	User's last name
email	VARCHAR(254)	NOT NULL	User's email address

Field	Data type	Constraint	Description
is_staff	BOOLEAN	NOT NULL	Whether the user can access the Django admin site
is_active	BOOLEAN	NOT NULL	Whether the account is active
date_joined	DATETIME	NOT NULL	Date and time the account was created
first_name	VARCHAR(150)	NOT NULL	User’s first name

Table: auth_group

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique group ID
name	VARCHAR(150)	NOT NULL, UNIQUE	Group name

Table: auth_permission

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique permission ID
Contenttypeid	INTEGER	NOT NULL; FK	Content type associated with the permission
codename	VARCHAR(100)	NOT NULL; UNIQUE	Permission codename
name	VARCHAR(150)	NOT NULL	Human-readable permission name

auth_group_permissions

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique permission record ID
Group_id	INTEGER	NOT NULL; FK → auth_group.id; UNIQUE with permission_id	Group receiving the permission
Permission_id	INTEGER	NOT NULL; FK → auth_permission.id; UNIQUE with group_id	Permission assigned to the group

Table: auth_user_groups

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique user-group record ID
user_id	INTEGER	NOT NULL; FK → auth_user.id; UNIQUE with group_id	User assigned to the group
group_id	INTEGER	NOT NULL; FK → auth_group.id; UNIQUE with user_id	Group assigned to the user

Table: auth_user_user_permissions

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique user-permission record ID
user_id	INTEGER	NOT NULL; FK → auth_user.id; UNIQUE with permission_id	User receiving the permission
permission_id	INTEGER	NOT NULL; FK → auth_permission.id; UNIQUE with user_id	Permission assigned directly to the user

Table: django_admin_log

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique admin-log record ID
object_id	TEXT	NULL	ID of the object changed in the admin site
object_repr	VARCHAR(200)	NULL	ID of the object changed in the admin site
action_flag	SMALLINT UNSIGNED	NOTNULL	Type of admin action
change_message	TEXT	NOTNULL	Description of the changes made
content_type_id	INTEGER	NULL; FK → django_content_type.id	Content type of the changed object
user_id	INTEGER	NOT NULL; FK → auth_user.id	Admin user who performed the action
action_time	DATETIME	NOT NULL	Date and time of the admin action

Table: django_content_type

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique content-type ID
app_label	VARCHAR(100)	NOT NULL; UNIQUE with model	Application label
model	VARCHAR(100)	NOT NULL; UNIQUE with app_label	Model name

Table: django_migrations

Field	Data type	Constraint	Description
id	INTEGER	PK, auto-increment	Unique migration record ID
app	VARCHAR(255)	NOT NULL	Application to which the migration belongs
name	VARCHAR(255)	NOT NULL	Migration name
applied	DATETIME	NOT NULL	Date and time the migration was applied

Table: django_session

Field	Data type	Constraint	Description
session_key	VARCHAR(40)	PK	Unique session key
session_data	TEXT	NOT NULL	Encoded session data
expire_date	DATETIME	NOT NULL	Session expiry date and time

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1.1 INTRODUCTION

Implementation is the stage of the project where theoretical designs are transformed into a functional system. During this phase, the primary workload shifts to the user department, often resulting in significant upheaval and impact on existing processes. If not carefully planned and controlled, implementation can lead to chaos and confusion.

This stage encompasses all activities required to transition from the old system to the new one. The new system may entirely replace an existing manual or automated system, or it may involve significant modifications to an existing setup. Proper implementation is crucial for providing a reliable system that meets organizational requirements. While successful implementation does not guarantee improvements, improper execution can hinder progress.

The implementation phase involves several key tasks:

- Careful Planning: Establishing a detailed roadmap for the implementation process.
- Investigation of System Constraints: Identifying limitations and challenges that may impact the transition.
- Design of Changeover Methods: Developing strategies to facilitate the transition to the new system.
- Staff Training: Ensuring users are well-trained in new procedures during the changeover.
- Evaluation of Changeover Methods: Assessing the effectiveness of the implemented strategies

5.1.2 CODING

Coding is the phase of a software development project where developers input the source code into a computer, which is then compiled into the final software program. Source code is written in high-level programming languages such as C#, Java, or Python, and is typically typed into an Integrated Development Environment (IDE). This source code is stored in text files on the computer.

Once the coding is complete, the text files are compiled into machine code, which consists of instructions that the computer can directly understand and execute. This phase is crucial, as it transforms design specifications into a functional software application.

CODING VALIDATION AND OPTIMIZATION

During the coding phase, validation ensures that data entered into forms is correctly added to the corresponding fields in the database tables. When the user presses the Submit button, controls appear on the form to confirm that the entered data is saved properly.

Key validation features Include:

- Data formatting: Lowercase letters are automatically converted to uppercase, ensuring consistency.
- Input Restrictions: Numeric values are not permitted in text boxes to maintain data integrity.

Validation serves as a critical check during the transition from theoretical designs to a working system. It helps reduce unnecessary loops in the program, enhancing efficiency.

Optimization is the final phase of the system development life cycle. It focuses on improving the performance of the code. An increase in the number of loops can lead to more executions, potentially causing the program to become unresponsive or "stuck." Therefore, careful optimization is essential to ensure smooth operation and responsiveness.

SAMPLE CODE:

```
import re
from django.shortcuts import render, redirect
from villa.models import query, user_details, builder_details,
    approved_builder_details
from builder_app.models import property_details
from user_app.models import booking

def validate_email_format(email):
    pattern = r'^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$'
    return bool(re.match(pattern, email))
def validate_phone_format(phone):
    clean_digits = re.sub(r'\D', '', phone)
    if clean_digits.startswith('91') and len(clean_digits) == 12:
        clean_digits = clean_digits[2:]
    if len(clean_digits) == 10:
        return True, f"+91 {clean_digits}"
    return False, None

def validate_password_format(password):
    if len(password) < 6:
        return False, "Password must be at least 6 characters long."
    if not re.search(r'[A-Z]', password):
        return False, "Password must contain at least one uppercase
    letter."
    if not re.search(r'[0-9]', password):
        return False, "Password must contain at least one number."
    if not re.search(r'[!@#$%^&*(),.?":{}|<>\-_+=\/~\[\]]', password):
        return False, "Password must contain at least one special symbol
```

```
    (!@#$%^&* etc.)."
    return True, None
def home(request):
    return render(request, 'index.html')

def index(request):
    contact_success = False
    if request.method == 'POST':
        name = request.POST.get('name', '').strip()
        email = request.POST.get('email', '').strip()
        subject = request.POST.get('subject', '').strip()
        message = request.POST.get('message', '').strip()
        if name and email and message:
            queries = query(name=name, email=email, subject=subject,
    message=message)
            queries.save()
            contact_success = True
    return render(request, 'index.html', {'contact_success':
    contact_success})

def dashboardindex(request): if not request.session.get('admin'):
    return redirect('/login') no_of_users = user_details.objects.count()
    no_of_builders = approved_builder_details.objects.count()
    pending_builder = builder_details.objects.count() no_of_properties =
    property_details.objects.count() no_of_bookings =
    booking.objects.count() no_of_queries = query.objects.count()
data = approved_builder_details.objects.all().order_by('-id')
    rdata = builder_details.objects.all().order_by('-id')
    recent_bookings = booking.objects.all().order_by('-id')[:5]
    recent_queries = query.objects.all().order_by('-id')[:5]
# Category counts for visual charts
    villa_count =
property_details.objects.filter(property_category__icontains='villa').
    count()
    apartment_count =
    property_details.objects.filter(property_category__icontains='appartme
    nt').count() +
    property_details.objects.filter(property_category__icontains='apartmen
    t').count()
    penthouse_count =
    property_details.objects.filter(property_category__icontains='penthous
    e').count()
    condo_count =
    property_details.objects.filter(property_category__icontains='condo').
    count()
    other_count = max(0, no_of_properties - (villa_count + apartment_count
    + penthouse_count + condo_count))
return render(request, 'adminindex.html', {
        'usercount': no_of_users,
        'buildercount': no_of_builders,
        'reqbuilder': pending_builder,
        'unitcount': no_of_properties,
        'bookingcount': no_of_bookings,
        'querycount': no_of_queries,
        'builders': data,
        'rbuilders': rdata,
        'recent_bookings': recent_bookings,
        'recent_queries': recent_queries,
        'cat_villa': villa_count,
        'cat_apt': apartment_count,
```

```
        'cat_pent': penthouse_count,
        'cat_condo': condo_count,
        'cat_other': other_count
    })
def login(request):
    if request.method == 'POST':
        email = request.POST.get('email', '').strip()
        password = request.POST.get('password', '').strip()
        if not email or not password:
            return render(request, 'login.html', {'error': 'Please
    enter both email and password.'})

        # 1. Admin login
        if email == 'admin@gmail.com' and password == 'admin':
            request.session['email'] = email
            request.session['admin'] = 'admin'
            return redirect('/dashboardindex')

        # 2. Regular user login
        user = user_details.objects.filter(email=email,
    password=password).first()
        if user:
            request.session['uid'] = user.id
            request.session['uname'] = user.name
            request.session['uemail'] = user.email
            request.session['user'] = 'user'
            return redirect('/')

        # 3. Approved builder login
        builder =
    approved_builder_details.objects.filter(buildemail=email,
    buildpassword=password).first()
        if builder:
            request.session['bid'] = builder.id
            request.session['bname'] = builder.buildname
            request.session['bemail'] = builder.buildemail
            request.session['builder'] = 'builder'
            return redirect('/')

        # 4. Check if builder is still pending approval
        if builder_details.objects.filter(buildemail=email,
    buildpassword=password).exists():
            return render(request, 'login.html', {'error': 'Your
    builder account is pending admin approval. Please check back later.'})

        return render(request, 'login.html', {'error': 'Invalid email
    or password. Please try again.'})

    return render(request, 'login.html')

def register(request):
    if request.method == 'POST':
        photo = request.FILES.get('photo')
        name = request.POST.get('name', '').strip()
        email = request.POST.get('email', '').strip()
        phone = request.POST.get('phone', '').strip()
        password = request.POST.get('password', '').strip()
        confirm_password = request.POST.get('confirm_password',
    '').strip()
if not name or not email or not phone or not password:
```

```
            return render(request, 'register.html', {'error': 'Please
    fill in all required fields.'})
        if not validate_email_format(email):
            return render(request, 'register.html', {'error': 'Please
    enter a valid email address (e.g. user@example.com).'})

        valid_phone, formatted_phone = validate_phone_format(phone)
        if not valid_phone:
            return render(request, 'register.html', {'error': 'Phone
    number must be exactly 10 digits.'})

        valid_pwd, pwd_error = validate_password_format(password)
        if not valid_pwd:
            return render(request, 'register.html', {'error':
    pwd_error})

        if password != confirm_password:
            return render(request, 'register.html', {'error':
    'Passwords do not match.'})

        if user_details.objects.filter(email=email).exists():
            return render(request, 'register.html', {'error': 'This
    email is already registered as a user.'})

        user_det = user_details(
            photo=photo,
            name=name,
            email=email,
            phone=formatted_phone,
            password=password
        )
        user_det.save()
        return redirect('/login')

    return render(request, 'register.html')

def buildregister(request):
    if request.method == 'POST':
        buildphoto = request.FILES.get('buildphoto')
        buildname = request.POST.get('buildname', '').strip()
        buildemail = request.POST.get('buildemail', '').strip()
        buildphone = request.POST.get('buildphone', '').strip()
        buildpassword = request.POST.get('buildpassword', '').strip()
        buildconfirm_password =
    request.POST.get('buildconfirm_password', '').strip()

        if not buildname or not buildemail or not buildphone or not
    buildpassword:
            return render(request, 'builderregister.html', {'error':
    'Please fill in all required fields.'})

        if not validate_email_format(buildemail):
            return render(request, 'builderregister.html', {'error':
    'Please enter a valid email address (e.g. builder@example.com).'})

        valid_phone, formatted_phone =
    validate_phone_format(buildphone)
        if not valid_phone:
            return render(request, 'builderregister.html', {'error':
    'Phone number must be exactly 10 digits.'})
```

```
        valid_pwd, pwd_error = validate_password_format(buildpassword)
        if not valid_pwd:
            return render(request, 'builderregister.html', {'error':
    pwd_error})

        if buildpassword != buildconfirm_password:
            return render(request, 'builderregister.html', {'error':
    'Passwords do not match.'})
if builder_details.objects.filter(buildemail=buildemail).exists() or
    approved_builder_details.objects.filter(buildemail=buildemail).exists(
    ):
            return render(request, 'builderregister.html', {'error':
    'This email is already registered as a builder.'})

        builder_det = builder_details(
            buildphoto=buildphoto,
            buildname=buildname,
            buildemail=buildemail,
            buildphone=formatted_phone,
            buildpassword=buildpassword
        )
        builder_det.save()
        return redirect('/login')

    return render(request, 'builderregister.html')
def logout(request):
    request.session.flush()
    return redirect('/')
```

CHAPTER 6
SYSTEM TESTING

6.1 INTRODUCTION

Software Testing is the process of executing software in a controlled manner, in order to answer the question - Does the software behave as specified? Software testing is often used in association with the term's verification and validation. Validation is the checking or testing of items, includes software, for conformance and consistency with an associated specification. Software testing is just one kind of verification, which also uses techniques such as reviews, analysis, inspections, and walkthroughs. Validation is the process of checking that what has been specified is what the user wanted.

Validation: Are we doing the right job? Verification: Are we doing the job right?

Software testing should not be confused with debugging. Debugging is the process of analyzing and localizing bugs when software does not behave as expected. Although the identification of some bugs will be obvious from playing with the software, a methodical approach to software testing is a much more thorough means for identifying bugs.

Other activities which are often associated with software testing are static analysis and dynamic analysis. Static analysis investigates the source code of software, looking for problems and gathering metrics without actually executing the code. Dynamic analysis looks at the behavior of software while it is executing, to provide information such as execution traces, timing profiles, and test coverage information.

BLACK BOX TESTING

Black box testing, also called behavioral testing, focuses on the functional requirements of software. This testing approach enables the software engineer to derive the input conditions that will fully exercise all requirements for a program. Black box testing attempts to find the errors like

- Incorrect or missing functions
- Interface errors
- Errors in data structures or external database access
- Behavior or performance errors
- Initialization and termination errors

In Black box testing software is exercised over a full range of inputs and outputs are observed for correctness.

WHITE BOX TESTING

White box testing is also called Glass box testing is a test case design control; structure of the procedural design to derive test cases using White box testing method, the software engineer can derive the test cases that guarantee that all independent paths within the module have been exercised at least once. Exercise all logic decisions on their true or false sides. Execute all loops at their boundaries and within their operational bounds. Exercise internal data structure to ensure their validity.

6.2 UNIT TESTING

In the unit test case will be testing the separate modules of the software. We will carry out black box testing where each module or component of software is tested individually. We will test the component by passing data through it and we will be monitoring data to find the errors. We will make sure that the component works without any troubles. The test primarily is carried out by the programmer who designed and implemented the module. Lead tester is carried out by the programmer who tests the modules to finalize the testing.

6.3 INTEGRATION TESTING

Integration testing involves combining different modules that have already been individually tested and verifying that they work together as a cohesive unit. This process ensures that all modules interact correctly and function seamlessly in conjunction with each other.During integration testing, it is crucial to address potential issues, such as:

- **Data Loss**: Data can be lost at interfaces between modules.
- **Adverse Effects**: One module may negatively impact another, leading to unexpected behaviors.
- **Subfunction Interactions**: When subfunctions are combined, they may not produce the desired outcomes for the overall function.

The goal of integration testing is to systematically uncover errors related to the interfaces between modules. By taking unit-tested modules and constructing a complete program structure, integration testing helps identify issues that could arise when modules interact.

Correcting errors discovered during this phase can be challenging, as the complexity of the entire program complicates the debugging process. Thus, thorough integration testing is essential to ensure that all components work together as intended.

6.4 USER ACCEPTANCE TESTING

User Acceptance Testing (UAT) is a critical phase of system validation that ensures the software meets user requirements in both simulated and live environments. This process begins with the software undergoing an initial phase known as **alpha testing**, where it is tested against simulated user requirements. During this phase, any errors or failures are identified and analyzed to ensure that the software behaves as expected.

Alpha testing typically involves a controlled group of users who evaluate the software in a testing environment. Their feedback is essential for refining the system before it is released for broader use. Following alpha testing, the software may move into a **beta testing** phase, where it is exposed to a wider audience to validate its functionality and usability in real-world scenarios.

UAT is vital for confirming that the software meets the needs and expectations of end users, ultimately ensuring a smooth transition to deployment.

CHAPTER 7
SYSTEM MAINTENANCE

7.1 INTRODUCTION

System maintenance is an important phase in the software development life cycle that ensures the developed system continues to function correctly after implementation. It involves monitoring the system, identifying errors, correcting defects, updating system components, and making necessary improvements according to changing requirements.

The Smart Property Management System requires regular maintenance to ensure that property information, customer details, builder information, site-visit bookings, wishlist data, and contact enquiries are managed accurately. Maintenance activities also help to preserve the reliability, security, and usability of the system.

The system is developed using the Django framework with a database-driven architecture. Therefore, maintenance includes checking the application, database, user authentication, session management, input validation, property images, and other implemented modules. Regular maintenance helps prevent errors and ensures smooth interaction between customers, builders, and administrators.

7.2 MAINTENANCE

Corrective Maintenance

Corrective maintenance involves identifying and fixing errors that may occur during the operation of the system. Errors related to user registration, login, property management, wishlist operations, site-visit scheduling, builder approval, and contact enquiries can be identified and corrected during maintenance.

Adaptive Maintenance

Adaptive maintenance involves modifying the system when changes occur in the operating environment or when new requirements are introduced. The Django application, database configuration, web environment, and supporting libraries may require updates to remain compatible with newer software environments.

Perfective Maintenance

Perfective maintenance focuses on improving the existing system based on user requirements. The system can be enhanced by improving the user interface, simplifying navigation, improving property search and display, and making administrative operations more convenient.

Preventive Maintenance

Preventive maintenance is carried out to reduce the possibility of future problems. Regular database backups, checking uploaded property images, reviewing validation rules, monitoring application errors, and keeping the software dependencies updated can help improve system reliability.

Database Maintenance

Database maintenance is required to ensure that property details, customer records, builder records, bookings, wish lists, and contact enquiries remain accurate and consistent. Unnecessary or incorrect records should be reviewed periodically, and regular backups should be maintained to prevent data loss.

Security Maintenance

Security maintenance ensures that only authorized users can access protected functions of the application. Session-based access control, password validation, input validation, and administrator access should be regularly reviewed. Security updates to the Django framework and other dependencies should also be applied when required.

CHAPTER 8
FUTURE ENHANCEMENT

8.1. INTRODUCTION

The present implementation provides property management and customer-interaction features, but the project title identifies a broader construction-monitoring objective. Future development can extend the existing platform while preserving the current property, user, builder, and booking records.

8.2. MERITS

- The existing Django structure provides a clear base for adding construction-related applications.
- Builder and property records can serve as starting points for construction project associations.
- The existing role-sensitive session design can be extended with more precise permissions.
- The existing dashboard can be expanded with construction progress indicators.
- The current media configuration can support construction progress photographs after suitable validation and storage improvements.

8.3. DEMERITS

- Construction stages, milestones, project schedules, and progress percentages are not currently implemented.
- There is no dedicated construction monitoring entity or workflow in the supplied models.
- The custom authentication implementation stores passwords directly and should not be used unchanged in production.
- The main project relationships are represented by identifier fields rather than database-level foreign keys.
- The supplied test modules do not contain implemented automated test cases.
- SQLite is suitable for an academic project but may require replacement or redesign for a large multi-user deployment.

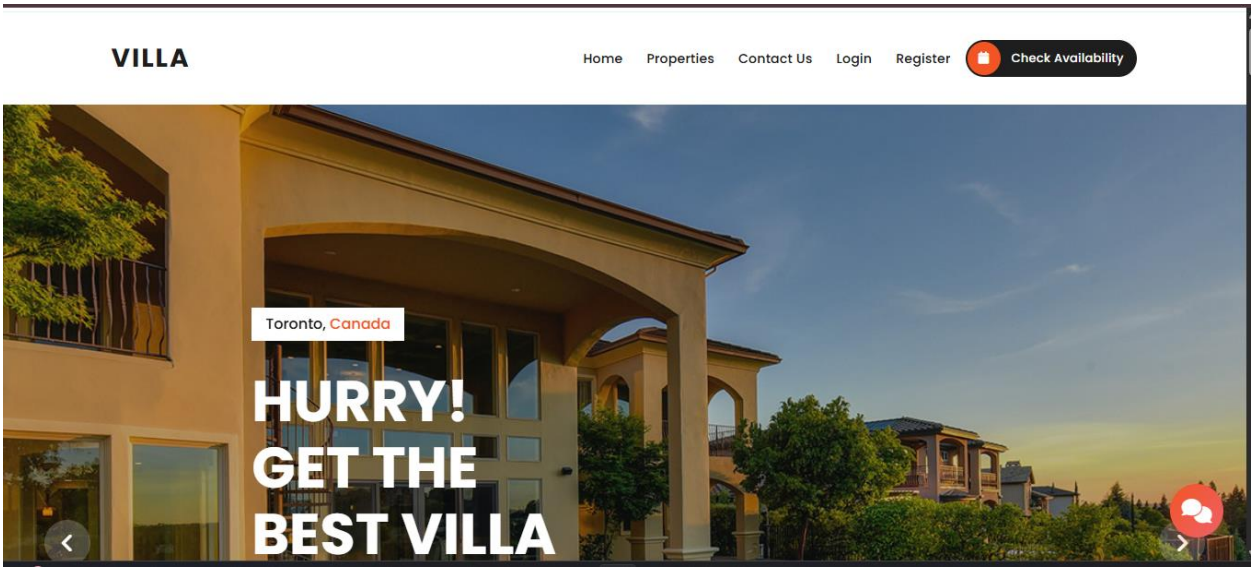
8.4. FUTURE ENHANCEMENTS

- Construction Project Module: Add construction-project records linked to approved builders and properties.

- Stage and Milestone Tracking: Add stages such as planning, foundation, structure, brickwork, services, plastering, finishing, and completion with dates and progress status.
- Progress Evidence: Allow authorized builders to upload dated progress photographs and supporting documents.
- Role-Based Construction Access: Add explicit permissions for administrators, builders, site personnel, and customers where required.
- Notifications: Provide email or in-application notifications for booking updates and construction milestones.
- Secure Authentication: Replace direct password storage with Django's password hashing and authentication framework.
- Relational Integrity: Replace identifier-only relationships with appropriate ForeignKey fields and migrations.
- Production Database: Consider PostgreSQL or another production-grade database for higher concurrency and deployment requirements.
- Testing Automation: Add model, view, authentication, upload, integration, and end-to-end tests and execute them in a configured Django environment.
- Reporting and Analytics: Extend the existing dashboard with reliable property, booking, builder, and future construction-progress analytics.

SCREENSHOTS

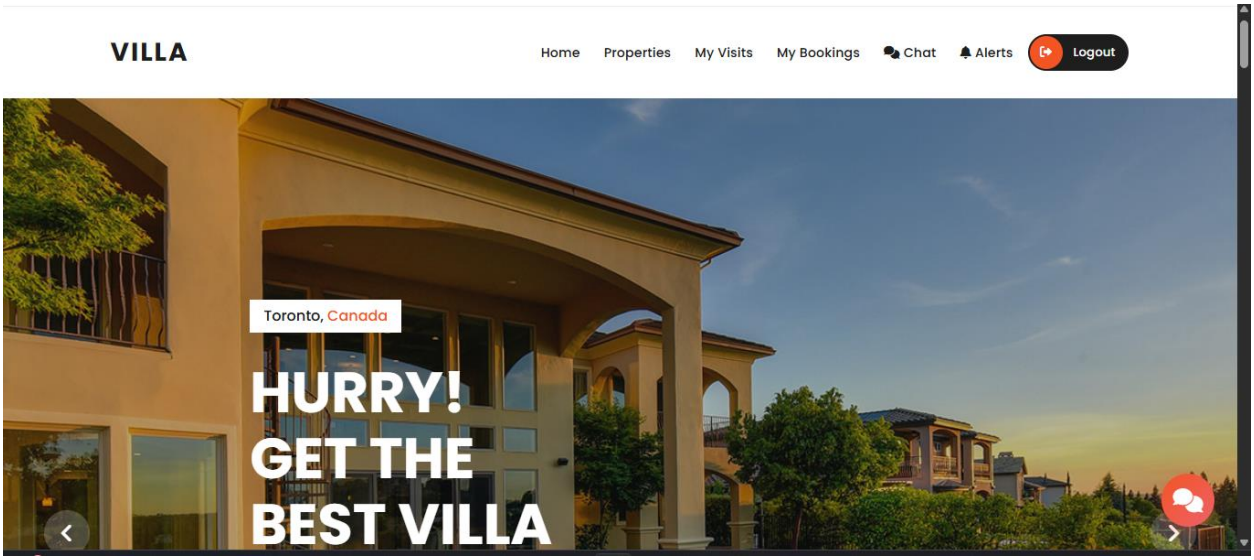
HOMEPAGE



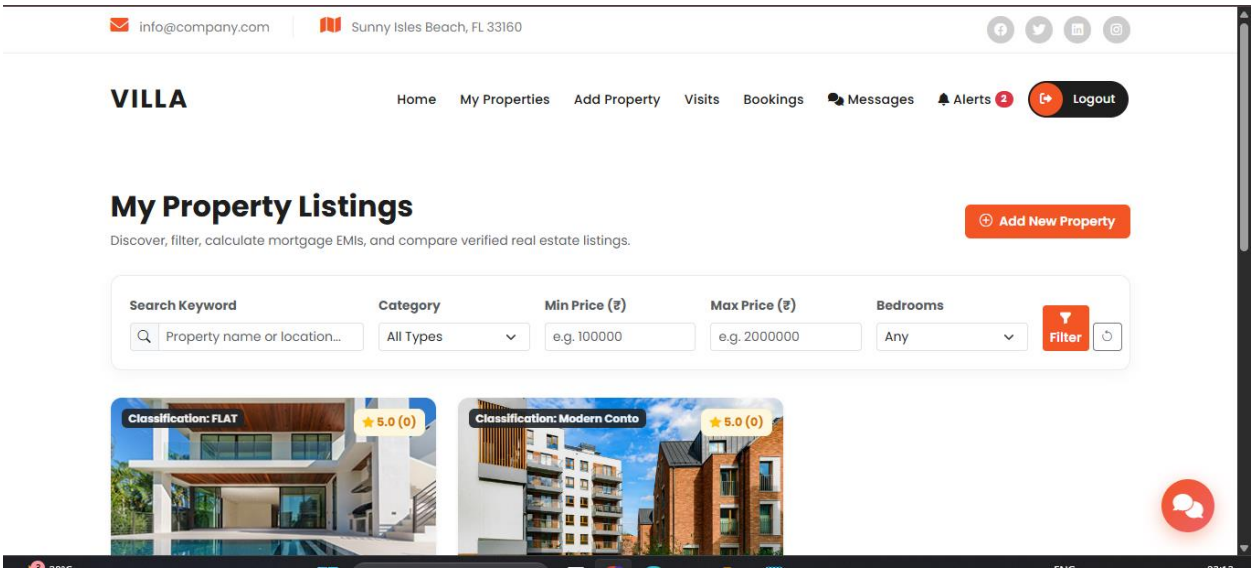
ADMIN PAGE



CUSTOMER PAGE



PROPERTY MANAGEMENT PAGE



BIBLOGRAPHY

Books Referenced:

- Roger S. Pressman, “Software Engineering: A Practitioner’s Approach”, McGraw-Hill Education.
- Ian Sommerville, “Software Engineering”, Pearson Education.
- Gary B. Shelly and Harry J. Rosenblatt, “System Analysis and Design”, Cengage Learning.
- Elmasri and Navathe, “Fundamentals of Database Systems”, Pearson Education.

WEB References:

- https://docs.djangoproject.com/
- https://www.python.org/doc/
- https://www.w3schools.com/html/
- https://www.w3schools.com/css/
- https://www.w3schools.com/js/
- https://www.sqlite.org/docs.html